OS → interface b/w user and hardware DATE

PROCESS MANAGEMENT

What is process? → a program in execution phase which can be created, executed & terminated.

System processes
↓
started by the OS itself.

user processes
↓
Processes that the user starts.

Why need of Process Management?

↳ Each process can run independently of other processes and have its own memory space. So process management is necessary to make sure that each process gets fair share of resources (memory, CPU time etc.) while ensuring that the system remains stable.

* Scheduling processes on CPU
* Creating and deleting processes
* Suspending and resuming processes
* Providing process synchronization
  ↓
  multiple processes are able to share same resources.

} Process management.

DATE

PROCESS CONTROL BLOCK (PCB)

↓

used to store info. about a process; keep track of all processes.

① Process state : A process goes through various states (new, ready, running, waiting, terminated). Process state defines current state of the process.

② Process ID : When a process is ~~is~~ created, a unique ID is assigned to the process which is identification of the process throughout.

③ Program counter : contains address of next instruction, which will be executed in the process

④ CPU Registers : specifies the registers used by a process.
↳ (hold data)

ex; general purpose registers, stack pointer etc.

⑤ Memory management information : info. related to memory limits & page table.
↓ how much memory is required.
↓ a table where OS stores mappings of logical addresses to physical addresses.

classmate

PAGE

DATE

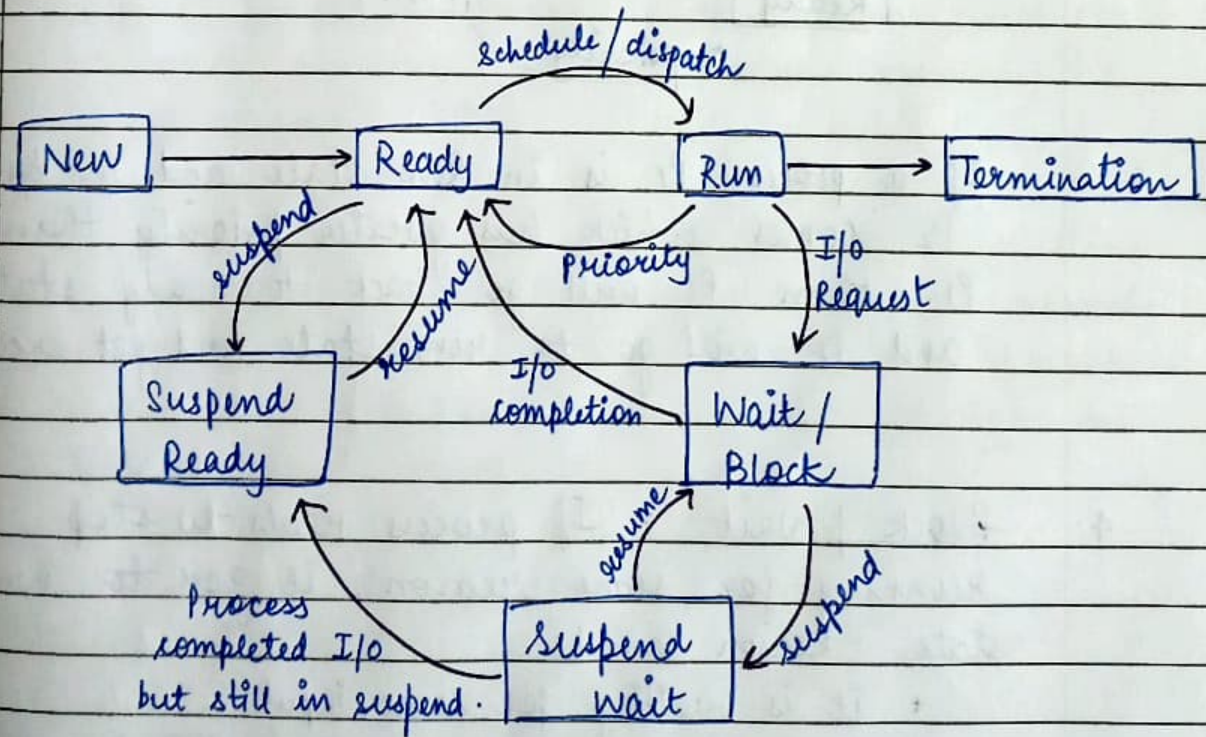
(6) Accounting information : Amount of CPU used for process execution, time limits etc.

(7) List of open files : contains record of files that are present in main memory during process execution

(8) List of open devices : contains list of all devices used in execution.

(9) Priority : Every process has diff. priority. higher priority ⇒ pehle execution.

PROCESS STATES



* New, Ready, Run, Terminated, Waiting → present in main memory
* suspend ready, suspend wait → secondary memory.

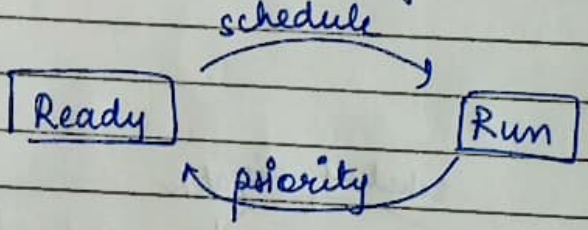
classmate PAGE

DATE

1. **New**: Process going to be picked up by the OS into the main memory.

2. **Ready**: whenever a process is created, it goes to ready state where it waits for the CPU to be assigned. Ready state can have multiple processes.

3. **Run**: One process (out of multiple) from ready state will be chosen depending upon the scheduling algorithm and it will go to run state for its execution.



If a process P_1 is in Run state and in b/w P_2 comes which has greater priority than P_1, then P_1 will go back to ready state and P_2 will go to run state and get executed.

4. **Block | Wait**: If process needs to stop running for some reason, it goes to waiting state. Reason can be:
   * it is waiting for some input.
   * it is waiting for a resource that is not available yet.

5. Termination : When process execution is complete, it goes to termination state.

6. Suspend ready : A process which was in ready state (main memory) is moved to secondary memory, then it is called suspend ready.

ex; if main memory is full and high priority process comes for execution, then we need to move lower priority process to secondary memory (suspend ready state) to make space for high priority process.

7. Suspend wait : moving process from wait state (main memory) to secondary memory i.e suspend wait state.

ex, a process is in wait state and it is waiting for a resource to get available and main memory gets full. so it is better to move it to suspend ready i.e secondary memory to make space for higher priority processes.

PROCESS OPERATIONS

1. Creation : Process can be created after request from user or a system call by a running process.

DATE

2. Scheduling: If a process is ready to be executed, it is in ready state. Selecting a process to get executed by CPU is called scheduling.

3. Execution: Execution means CPU has been assigned. During execution, it can go to wait / block state.

4. Killing: After process execution, OS will terminate / kill the process.

SCHEDULING → allows processes to use CPU alongwith making the system more efficient, faster.

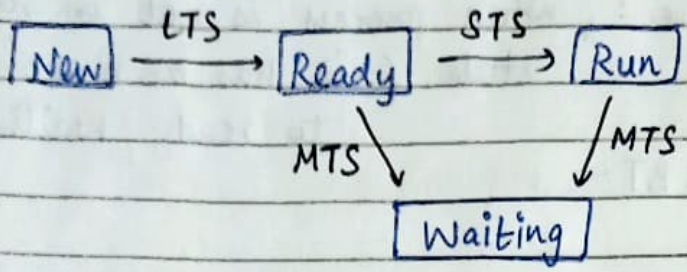
Types of process schedulers

- Long-term scheduler (LTS) → admits processes to ready state from new state.
- Medium term scheduler (MTS) → responsible for suspending and resuming processes in between.
- short term scheduler (STS) → selects one process from ready state and moves it to run state for execution.

classmate

PAGE

DATE



CPU SCHEDULING ALGORITHMS

- Keep CPU as busy as possible
- Allocation of CPU must be fair.
- Minimum waiting time for a process.
- Time taken by a process for execution is least.

Terminologies :

→ **Arrival time** (AT) : Time at which process arrives in ready state.

→ **Burst Time** (BT) (CPU Time) : how much time process needs for execution.

→ **Completion Time** (CT) : Time at which process completes execution

→ **Turn-around time** (TAT) : Time interval b/w time of process arrival and its completion.

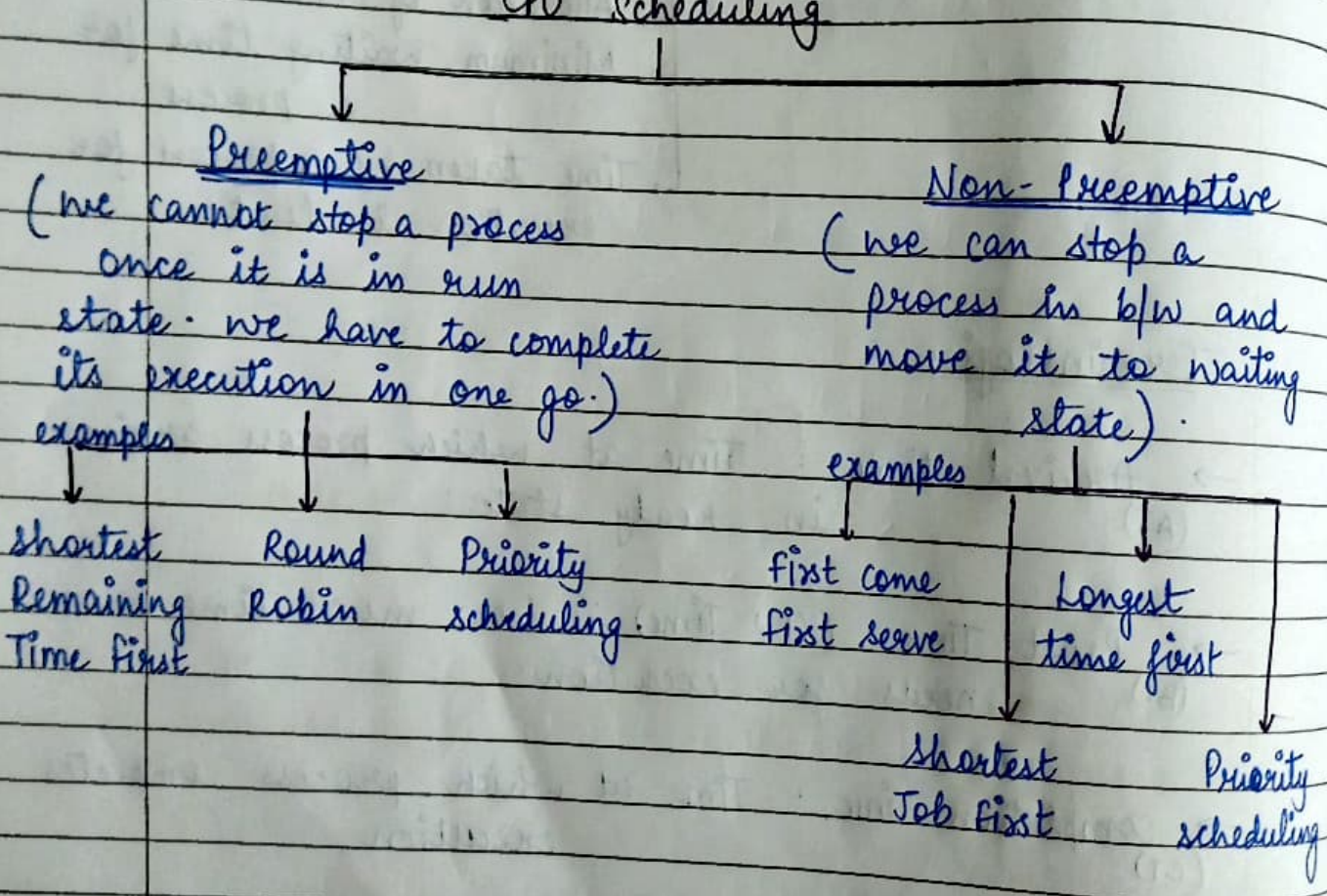
$$TAT = CT - AT$$

DATE

→ Waiting Time : time when process is not in run state (it has been moved to ready / waiting state)
(WT)

WT = TAT - BT.

→ Response Time : Pehli baar process ko CPU kab mila.
(RT)



① First Come First Serve (FCFS)

↳ Jo pehle aaya vo execute hoga.

~~WT RT AT.~~

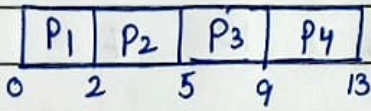
DATE

mode : non preemptive (process ko beechme rok nahi sakte)

criteria : arrival time.

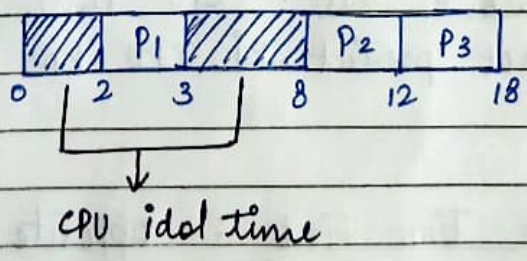
	AT (given)	BT (given)	CT	RT	TAT	WT
P_1	0	2	2	0	2	0
P_2	2	3	5	2	3	0
P_3	3	4	9	5	6	2
P_4	4	4	13	9	9	5

Gant chart:



	AT	BT	CT	RT	TAT	WT
P_1	2	1	3	2	1	0
P_2	8	4	12	8	4	0
P_3	9	6	18	12	9	3.

Gant chart:



★ Disadvantage of FCFS : If there is a process that requires very long time to execute; so processes that need less time have to wait till it completes its execution. This is called starvation / convoy effect.

classmate

PAGE

DATE

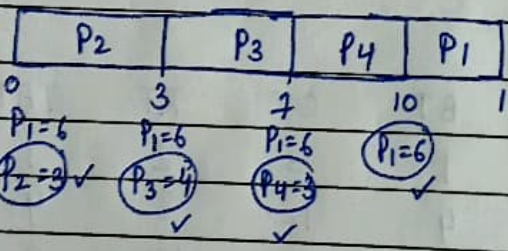
② Shortest Job first (SJF)

Process which has smallest execution time (BT) gets executed first.

- criteria : burst time (kam wala pehle)
- mode : non-preemptive.

	AT	BT	CT	RT	TAT	WT
P1	0	6	16	10	16	10
P2	0	3	3	0	3	0
P3	2	4	7	3	5	1
P4	6	3	10	7	4	1

Gant chart :



★ Drawback of SJF : CPU time is not fixed. We cannot predict ki CPU ko kitna time lagega.

③ Longest Time first : opposite of SJF

- criteria : burst time (zyada wala pehle)
- mode : non preemptive

DATE

④ Shortest Remaining Time First (SRTF) :

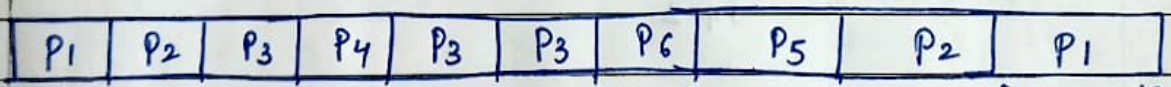
↓

select process which has shortest remaining time left to execute.

mode: preemptive (process ko beechme rok sakte hai)

	AT	BT	CT	RT	TAT	WT
P1	0	7	19	0	19	12
P2	1	5	13	1	12	7
P3	2	3	6	2	4	1
P4	3	1	4	3	1	0
P5	4	2	9	7	5	3
P6	5	1	7	6	2	1

Gant chart :



P1=7 P1=6 P1=6 P1=6 P1=6 P1=6 P1=6 P1=6 P1=6
P2=5 P2=4 P2=4 P2=4 P2=4 P2=4 P2=4 P2=4
P3=3 P3=2 P3=2 P3=1 P5=2 P5=2
P4=1 P5=2 P5=2 P6=1
P6=1

$$\text{Avg. WT} = \frac{12+7+1+0+3+1}{6} = \frac{4 \text{ sec}}{}$$

DATE

⑤ Priority scheduling :

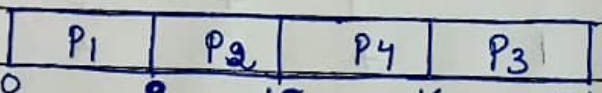
{ criteria : priority
{ mode : preemptive / non preemptive

① non preemptive mode

4 = highest priority]
1 = lowest priority]

	AT	P	BT	CT	RT	TAT	WT
P1	0	4	8	8	0	8	0
P2	0	3	4	12	8	12	8
P3	4	1	5	21	16	17	12
P4	5	2	4	16	12	11	7

Gant chart :



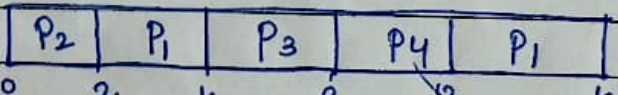
✓P1=4 pri. ✓P2=3 ✓P3=1
✓P2=3 pri. P3=1 ✓P4=2
P4=2

② preemptive mode

1 = highest
4 = lowest

	AT	P	BT	CT	RT	TAT	WT
P1	0	4	~~8~~ 6	19	2	19	11
P2	0	3	2	2	0	2	0
P3	4	1 (H)	5	9	4	5	0
P4	5	2	4	3	9	8	4

Gant chart :



P1=4 P1=4 P1=4 P1=4
P2=3 ✓P3=1 ✓P4=2

classmate

PAGE

DATE

⑥ Round Robin (RR)

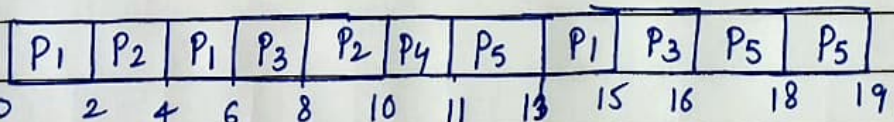
↳ each process is executed for a given time slot (called time quantum).

mode: preemptive.

TQ = 2.

	AT	BT	CT	RT	TAT	WT
P1	0	~~6 4 2~~ 0	15	0	15	9
P2	1	~~4 2~~ 0	10	2	9	5
P3	3	~~3 1~~ 0	16	6	13	10
P4	5	~~1~~ 0	11	10	6	5
P5	5	~~5 3 1~~ 0	19	11	14	9

Ready state → ~~P1 P2 P1 P3 P2 P4 P5 P1 P3 P5~~ P5

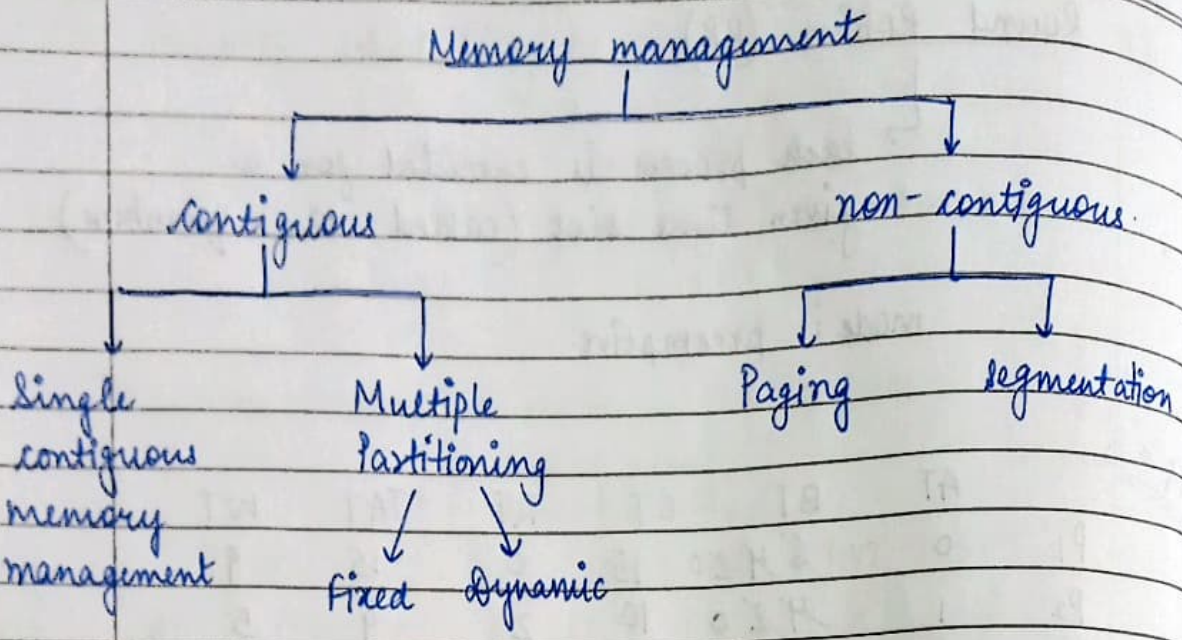


MEMORY MANAGEMENT → task of dividing memory among different processes

Why memory management?

- allocate & de-allocate memory before and after process execution
- keep track of memory used by processes
- minimize fragmentation issues
- maintain data integrity.

DATE



① Contiguous Memory management

→ each program occupies single block of storage location i.e set of memory location with continuous addresses.
(best example → array.)

single contiguous

* main memory is divided into two partitions. OS is present in one partition and user processes in other partition.

disad * can't handle multiple programs.

multiple partitioning.

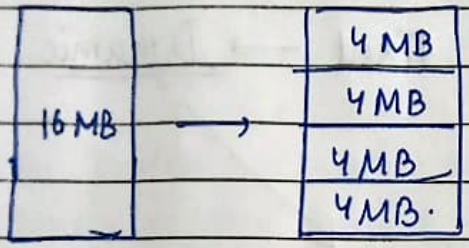
* main memory is divided into multiple parts in order to load multiple processes simultaneously.

PAGE

DATE

Multiple Partitioning

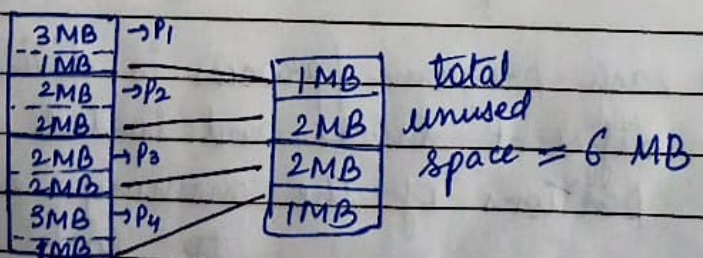
- **FIXED**
  - All Partitions are of fixed size (same or different).
- **DYNAMIC**
  - Partition size not decided initially. Size of each partition is equal to size of process.



★ Disadvantages of fixed Partitioning:

1. Internal fragmentation.
   - ↳ If size of process < size of partition, there is a lot of unused space.

$P_1 = 3$ MB
$P_2 = 2$ MB
$P_3 = 2$ MB
$P_4 = 3$ MB



classmate

PAGE

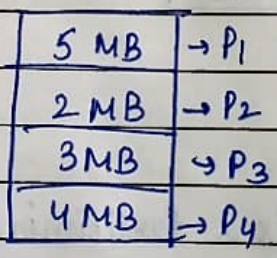
DATE

2. External fragmentation

↳ If we get a new process $P_5 = 4MB$; although we have total 6 MB unused; but we can't allocate it to P_5; as it is not in contiguous form.

→ Overcome disadv. of fixed → Dynamic.

$P_1 = 5MB$
$P_2 = 2MB$
$P_3 = 3MB$
$P_4 = 4MB$



~~partition size~~
= process size.

(2) Non-contiguous memory management

→ each program / process is divided into different blocks and loaded at different portions of the memory (need not adjacent to each other).

DATE

In the form of Pages

① PAGING : Retrieving processes from secondary storage into the main memory.

→ Divide / separate a process into pages
→ Divide / separate main memory into frames.

page size = frame size.

* pages → logical address
* frames → physical address.

When process requests memory, OS allocates frames (from main memory) to the process and maps process's logical pages to memory's physical frames.

Mapping b/w physical frames and logical pages is maintained by ~~mapper~~ page table.

→ LAS (Logical address space) = set of all logical addresses
→ PAS (Physical address space) = set of all physical addresses.

example, Physical address space = 128 KB
Logical address space = 256 MB
Page / frame size = 4 B.

$$\begin{cases} 1\ KB = 2^{10}\ B \\ 1\ MB = 2^{20}\ B \\ 1\ GB = 2^{30}\ B \end{cases}$$

DATE

$$PAS = 128\ KB = 2^7 \times 2^{10}\ B = 2^{17}\ B$$

$\Rightarrow$ physical address is of 17 bits total.

$$LAS = 256\ MB = 2^8 \times 2^{20}\ B = 2^{28}\ B$$

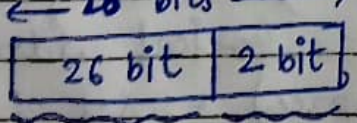
$\Rightarrow$ logical address is of 28 bits total.

Page / Frame size $= 4B = 2^2\ B$

$$\text{No of frames} = \frac{PAS}{\text{size}} = \frac{2^{17}}{2^2} = 2^{15}$$

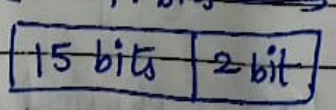
$$\text{No. of pages} = \frac{LAS}{\text{size}} = \frac{2^{28}}{2^2} = 2^{26}.$$

Logical address



page no. page offset

Physical address



(2) SEGMENTATION : memory is divided into variable size parts. Each part is known as segment which can be allocated to a process.

DATE

Why Segmentation?

↳ Paging divides all process in the form of pages without taking into consideration that there can be some parts of a process which need to be loaded in same page. So paging does not take user's view into account; but segmentation does.

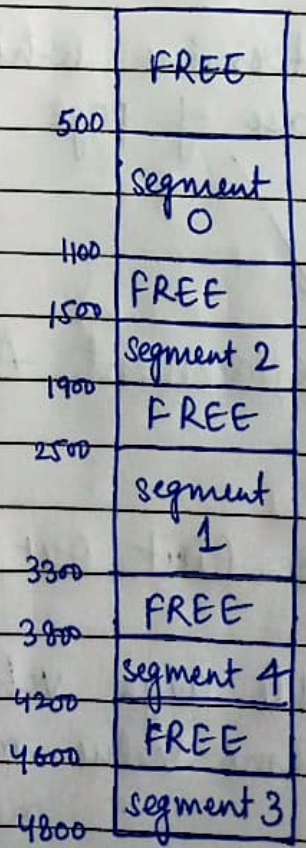
→ Info. about each segment contained in segment table.

- Base address (starting physical address of the segment)
- Limit / offset (Length of segment).

Segment table

	Base Add.	Limit
0	500	600
1	2500	800
2	1500	400
3	4600	200
4	3800	400

→



DATE

VIRTUAL MEMORY

↳ secondary memory is treated as a part of main memory (hypothetically).

→ **Demand Paging** : Pages of a process which are least used ; get stored in secondary memory.

⇒ Page is copied to the main memory whenever its demand is made or any page fault occurs.

↳ when a process needs to access data within its address space ; but it fails to load.

To determine which pages will be replaced in case of page fault.

↓

PAGE Replacement Algorithms

1. **First In First Out (FIFO)** :

Jo pehle aaya vohi replace kardiya.
Agar same value milgayi = hit
" " " nahi mili = fault & replace.

(F = fault, H = hit)

(Hit Ratio = no. of hits / Total pages.)

DATE

F	F	F	H	F	F	F	F	F	H	H	F	F
2	1	0	0	3	4	5	1	0	5	1	3	2

Page Page size = 4.

~~2~~	~~4~~	3
1	~~5~~	2
~~0~~		1
~~3~~		0

Hit = 3

Fault = 10.

(LRU)

2. Least Recently used: Replace the frame jo least recently use hua ho. (~~Don't~~ Avoid repetition).

✓(2)103 ✓(1)034 ✓(0)513

2	1	0	0	3	(4)	(5)	1	0	5	1	3	(2)
F	F	F	H	F	F	F	F	F	H	H	F	F

~~2~~	~~4~~	3
1		5
~~0~~		1
~~3~~		0 2

✓(7)120 ✓(1)230

7	0	1	2	0	(3)	0	(4)	2	3	0	3
F	F	F	F	H	F	H	F	H	H	H	H

~~7~~	3
0	
1	4
2	

(F = fault, H = hit).

(Hit Ratio = No. of hits / Total pages.)

DATE

F	F	F	H	F	F	F	F	F	H	H	F	F
2	1	0	0	3	4	5	1	0	5	1	3	2

Page size = 4.

2	4 3
1	5 2
0	1
3	0

Hit = 3

Fault = 10.

(LRU)

2. Least Recently used : Replace the frame jo least recently use hua ho. (Don't short) (Avoid repetition)

(2) 1 0 3 (1) 0 3 4 (5) 1 3

2	1	0	0	3	4	5	1	0	5	1	3	2
F	F	F	H	F	F	F	F	F	H	H	F	F

2	4 3
1	5
0	1
3	0 2

(7) 1 2 0 (1) 2 3 0

7	0	1	2	0	3	0	4	2	3	0	3
F	F	F	F	H	F	H	F	H	H	H	H

7	3
0	
1	4
2	

discrete

PAGE

4. **Most Recently used (MRU)** : opp. of least recently used. Replace frame jo sabse latest use hua ho.

DATE

3. **Optimal** : Replace that page jo in future for longest duration use nahi hone wala.

7 door door tak nhi hai — 1 nhi hai

7	0	1	2	0	3	0	4	2	3	0	3	2	3
F	F	F	F	H	F	H	F	H	H	H	H	H	H

~~7~~	3
0	
~~1~~	4
2	

SWAPPING : Process temporarily swapped from main to sec memory so that main memory can be made available for other processes. Process in sec. Memory is stored in 'swap space'.

- Swap-in (RAM to hard disk)
- Swap-out (hard disk to RAM).

THRASHING : Poor performance of virtual memory OR when same pages are being repeatedly loaded. Thrashing → constant page faults.

How to avoid? → increase amount of RAM; decrease no. of processes running on the computer; replace programs/pages.

DATE

FILE SYSTEM

↳ a medium used for saving and managing data in the computer. Stores, organizes and manages files and directories.

Types:

→ FAT (File Allocation Table): used by older versions of Windows and other OS.

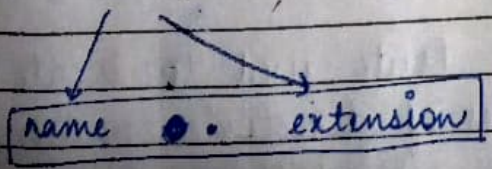
→ NTFS (New Technology File System): modern file system used by Windows. Supports file permissions, compression & encryption.

→ ext (Extended File System): commonly used in Linux and other Unix based systems.

→ HFS (Hierarchical file System): used by MacOS.

→ APFS (Apple file System): new file system introduced by Apple for their devices.

File ? → info. stored in secondary storage.



DATE

File Type	Extension	Function
Executable	exe, com, bin	Read to run machine language
Object	obj, o	compiled machine language
Source code	C, java	code of various languages
Batch	bat, sh	commands
Text	txt, doc	Textual docs.
Archive	arc, zip, tar	Related files grouped into one compressed file.

File System Hierarchy

- Root Directory → top level directory, represented by '/'
- Subdirectories → located within other directories

⇒ File path → Route used to locate files (series of directories names separated by slash)

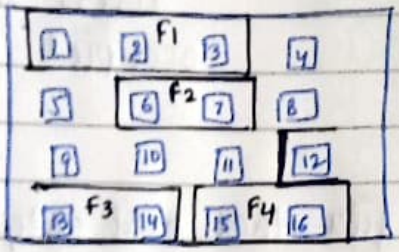
DATE

File Allocation Methods

↳ how files are stored and organized.

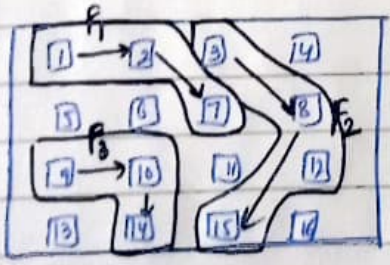
* **Contiguous allocation** : each file is allocated a contiguous block of storage space. (like array)

This method is quick, but it can lead to issues if we frequently add or delete files.



File	Start	Length
F1	0	3
F2	6	2
F3	12	3
F4	15	2

* **Linked Allocation** : each file is a linked list of blocks (need not be contiguous). Each block has a pointer to next block.

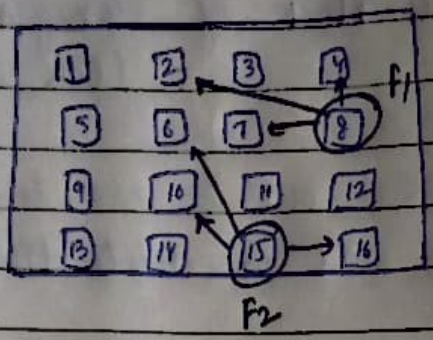


File	Blocks
F1	1 → 2 → 7
F2	3 → 8 → 15
F3	9 → 10 → 14

Disadv : we cannot directly access any file. We need to access every block individually + Pointers take extra space.

DATE

* Indexed Allocation : A special block known as index block contains pointers to all blocks occupied by a file. Each file has its own index block.



File	Index block
F1	8
F2	15

* supports direct access to blocks occupied by file.

Disadv : For small files that need only 2-3 blocks, one entire block i.e index block is wasted to keep the pointers of the file. This can lead to ineffective memory utilization.

PAGE